

CÍVICA SOFTWARE

Sistema de gestión de incidencias · Turno de guardia

Cuadernillo del desarrollador de turno

Prueba práctica del Corte 1 · Algoritmos y estructura de datos

Nombre completo	Andres Felipe Ortegon Porras	Fecha	31/08/2026
-----------------	------------------------------	-------	------------

Este cuadernillo contiene todo lo que usted necesita durante el turno: la mecánica de la prueba, las instrucciones detalladas de cada uno de los cinco tickets, y su hoja de ruta. Consérvelo: es el documento que se entrega en la mesa de despacho al cerrar el turno.

Cómo funciona el turno

Usted acaba de recibir el turno de guardia. Hay cinco tickets abiertos, ordenados de menor a mayor severidad: desde un P4, que es una revisión de código de rutina, hasta un P0, que significa producción caída y gente esperando. Su trabajo es cerrar los que pueda en 100 minutos.

El ciclo de trabajo

1. **Lea el ticket completo:** el reporte de quien lo abrió y el criterio de aceptación. El criterio dice exactamente qué se considera resuelto.
2. **Corrija únicamente el archivo indicado** en la ficha del ticket.
3. **Ejecute las pruebas automáticas** del ticket. Le muestran qué verificaciones pasan y cuáles fallan. Puede ejecutarlas las veces que quiera: no hay penalización por intentar.
4. **Cuando todas pasen, aparece el CÓDIGO DE CIERRE.** Anótelos en su hoja de ruta, al final de este cuadernillo.

SOBRE EL CÓDIGO DE CIERRE No está escrito en ningún archivo: se calcula a partir de los resultados correctos. No se puede adivinar, ni leer del archivo de pruebas, ni copiar del compañero, porque un resultado equivocado produce un código distinto.

Ejemplo del ciclo

Este ejemplo no corresponde a ningún ticket real; sirve para que reconozca el formato.

Primera ejecución: todavía falla

```
$ python3 pruebas_ejemplo.py
TCK-0000 · verificacion
[FALLA] total_por_dia: obtuvo [16, 20, 21, 14]
        esperaba [15, 11, 14, 7, 12, 12]
[FALLA] dia_mas_flojo: obtuvo None
```

Ticket ABIERTO. Corrija y vuelva a ejecutar.

Después de corregir: aparece el código

```
$ python3 pruebas_ejemplo.py
TCK-0000 · verificacion
[OK ] total_por_dia: obtuvo [15, 11, 14, 7, 12, 12]
[OK ] dia_mas_flojo: obtuvo 3
```

TICKET CERRADO – código de cierre: 0000-DEMO

Ese **0000-DEMO** es lo que usted copia en su hoja de ruta. Cada ticket tiene el suyo, con formato distinto.

Sus archivos y sus comandos

En su equipo hay una carpeta llamada prueba_corte1 con una subcarpeta por ticket:

```
prueba_corte1/estudiante/
TCK-4420/    ocupacion.py    pruebas_t2.py
TCK-4421/    inventario.cpp
TCK-4422/    catalogo.cpp    pruebas_t4.sh
TCK-4423/    fila.py        pruebas_t5.py
```

El comando exacto de cada ticket

Ticket	Usted modifica	Usted ejecuta
TCK-4419	— (es en papel)	—
TCK-4420	ocupacion.py	python3 pruebas_t2.py
TCK-4421	inventario.cpp	g++ -std=c++17 -fsanitize=address -g -o inventario inventario.cpp ./inventario
TCK-4422	catalogo.cpp	./pruebas_t4.sh (o bien: bash pruebas_t4.sh)
TCK-4423	fila.py	python3 pruebas_t5.py

REGLA QUE NO TIENE EXCEPCIÓN Los archivos cuyo nombre empieza por «pruebas» NO se modifican. Están ahí para que usted se verifique solo. Alterarlos para que impriman el código es fraude y aplica el protocolo del reglamento (sección 13 de la guía).

Su nota y cuándo puede salir

La nota es la suma de los tickets que cierre. Los tickets son independientes: no hay candado, puede resolverlos en cualquier orden y saltarse el que se le atragante.

Puede cerrar su turno y salir cuando quiera. No tiene que esperar a nadie ni quedarse los 100 minutos.

Ticket	Sev	Vale	Si cierra hasta aquí y sale	Resultado
TCK-4419	P4	0.7		
TCK-4420	P3	0.9	TC(8:25) 0.9	Aprueba
TCK-4421	P2	1.0	TC(9:10) 1.0	Aprueba
TCK-4422	P1	1.1		
TCK-4423	P0	1.3		

- **El P0 vale más que los demás.** Si le queda poco tiempo y no ha cerrado nada, ese es el que más rinde.
- **Las ventanas de tiempo son una referencia de ritmo, no un plazo.** Si cierra un ticket después de que su ventana pasó, cuenta igual.
- **La decisión de salir es suya.** Salir con lo que tiene o arriesgar tiempo por más alcance es la misma decisión que se toma en un equipo de desarrollo cuando se acerca una entrega.

La mesa de despacho

1. Cuando decida cerrar su turno, lleve este cuadernillo a la mesa.
2. La profesora le va a pedir que ejecute las pruebas en su pantalla, delante de ella. No sirve una captura ni una foto.
3. Coteja los códigos, firma su hoja de ruta y usted se retira.

Al salir, no comente los tickets en el pasillo hasta que termine la hora.

Reglas del turno

- Trabajo individual. Cada uno en su equipo.
- Material permitido: únicamente la documentación oficial de C++ y de Python. Nada más abierto.
- Puede ejecutar las pruebas cuantas veces quiera.
- Si su código no compila, lea el error completo antes de levantar la mano.
- Una vez firma en la mesa, el turno terminó: no se puede volver a entrar.

TCK-4419 Revisión de código de un compañero**SEVERIDAD P4** · Interno · ventana 0–15 min · En papel · sin computador · 0.7 puntos

Reportado por: Solicitud de integración pendiente de aprobación

«Antes de aprobar mi pull request, ¿me la revisas? Son seis fragmentos cortos.»

Qué debe hacer

Para los fragmentos 1 a 5, escriba EXACTAMENTE qué imprime cada uno. Para el fragmento 6, escriba qué está mal en el código.

Este ticket se resuelve sin computador y se responde en este mismo cuadernillo. Se califica después de la prueba, así que no hay código de cierre.

IMPORTANTE Este es el único ticket que no depende del computador. Si tiene un problema técnico con su equipo, este ticket le garantiza que no sale en cero. Respóndalo completo.

Fragmento 1 · C++ — ¿qué imprime?

```
int v[4] = {5, 3, 9, 1};
int* p = v;
cout << *(p + 2);
```

9

Fragmento 2 · Python — ¿qué imprime?

```
m = [[0] * 3] * 2
m[0][0] = 7
print(m)
```

[[7,0,0],[7,0,0]]

Fragmento 3 · Python — ¿qué imprime?

```
def f(lista):
    lista = lista + [4]

datos = [1, 2, 3]
f(datos)
print(datos)
```

[1, 2, 3] + [4]

Fragmento 4 · Python — ¿qué imprime?

```
def g(lista):
    lista.append(4)

datos = [1, 2, 3]
g(datos)
print(datos)
```

[1, 2, 3, 4]

Fragmento 5 · C++ — ¿qué imprime?

```
class A { public: string quien() const { return "Soy A"; } };
class B : public A { public: string quien() const { return "Soy B"; } };

B objeto;
A* p = &objeto;
cout << p->quien();
```

Soy B

Fragmento 6 · C++ — ¿qué está mal?

```
class Registro {
    int* datos;
public:
    Registro(int n) { datos = new int[n]; }
    ~Registro() { }
};
```

No esta private en el código antes de int* datos, ya que en las clases en c++ se necesitan saber si es public o private.
También porque es un atributo y los atributos deben estar en private, tambein falta el delete datos

TCK-4420 El reporte semanal entrega datos errados**SEVERIDAD P3** · RedAcopio · ventana 15–35 min · Python · 0.9 puntos

Reportado por: Coordinación de la central de reciclaje

«El informe de totales por punto de acopio sale bien, pero el de totales por día está mal: nos da los mismos números que el de puntos.»

«Además necesitamos dos datos que nunca se implementaron: cuál fue el día de menor recolección y cuántos registros quedaron en cero porque el punto no operó.»

Contexto del sistema

El archivo `ocupacion.py` contiene una matriz donde las filas son puntos de acopio y las columnas son días de la semana. Cada celda es lo recogido ese día en ese punto. La matriz y el archivo de pruebas no se modifican.

Qué debe hacer

1. **Corregir `total_por_dia(m)`**. Debe devolver una lista con el total de cada COLUMNA (cada día). Actualmente devuelve los totales por fila. Revise cuál de los dos ciclos va por fuera.
2. **Implementar `dia_mas_flojo(m)`**. Debe devolver el ÍNDICE (no el valor) del día con menor recolección total.
3. **Implementar `puntos_inactivos(m)`**. Debe devolver cuántos registros de la matriz están en cero.

Criterio de aceptación

- Las cuatro verificaciones de `pruebas_t2.py` en OK.
- No cambie los nombres ni los parámetros de las funciones: el archivo de pruebas las llama por su nombre.
- No modifique la matriz `ocupacion`.

Cómo se verifica

```
$ cd prueba_corte1/estudiante/TCK-4420
$ python3 pruebas_t2.py
```

Errores frecuentes en este ticket

- Devolver el valor mínimo en vez del índice, en `dia_mas_flojo`.
- Contar filas o columnas en cero en vez de registros individuales, en `puntos_inactivos`.
- Dejar un `return` dentro del ciclo y cortar el recorrido antes de tiempo.

Código de cierre obtenido: _____

```
def total_por_dia(m):
    """Devuelve una lista con el total recogido cada día (columna).
    |   BUG REPORTADO: entrega totales incorrectos."""
    totales = []

    for j in range(len(m[0])): # recorre columnas
        s = 0 # suma de la columna j
        for i in range(len(m)): # recorre filas
            s += m[i][j] # suma el valor de la fila i, columna j
        totales.append(s) # agrega el total de la columna j a la lista de totales
    return totales

def dia_mas_flojo(m):
    """Devuelve el índice del día con MENOR recolección total.
    |   PENDIENTE: implementar."""

    totales = total_por_dia(m) # obtiene los totales por día
    menorTotal = totales[0] # inicializa el menor total con el primer día
    indiceMenor = 0 # inicializa el índice del día con menor total con el primer día

    for i in range(1, len(totales)): # recorre los totales desde el segundo día
        if totales[i] < menorTotal: # si el total del día i es menor que el menor total actual
            menorTotal = totales[i] # actualiza el menor total
            indiceMenor = i # actualiza el índice del día con menor total

    return indiceMenor # devuelve el índice del día con menor recolección total

def puntos_inactivos(m):
    """Devuelve cuantos registros están en 0 (el punto no operó ese día).
    |   PENDIENTE: implementar."""
    inactivos = 0 # inicializa el contador de puntos inactivos
    for fila in m: # recorre cada fila (punto de acopio)
        for v in fila: # recorre cada valor en la fila (días de la semana)
            if v == 0: # si el valor es 0, significa que el punto no operó ese día
                inactivos += 1 # incrementa el contador de puntos inactivos
    return inactivos # devuelve el total de puntos inactivos
```

TCK-4420 · verificación

```
[OK ] total_por_punto: obtuvo [16, 20, 21, 14]
[OK ] total_por_dia: obtuvo [15, 11, 14, 7, 12, 12]
[OK ] dia_mas_flojo: obtuvo 3
[OK ] puntos_inactivos: obtuvo 5
```

TICKET CERRADO – código de cierre: 4420-7135

PS C:\Users\Usuario\OneDrive\Desktop\Primer parcial practico> █

TCK-4421 El servicio consume memoria sin parar**SEVERIDAD P2** · PrestaLab · ventana 35–55 min · C++ · 1.0 puntos**Reportado por: Infraestructura**

«El proceso de inventario arranca en 40 MB y a las seis horas va en 900 MB. Hay que reiniciarlo todas las noches.»

«Y desde el martes, además, el total de uso del equipo EQ-03 sale en 7 cuando debería ser 8.»

Contexto del sistema

El archivo inventario.cpp gestiona equipos de laboratorio. Cada objeto Equipo reserva memoria dinámica para su historial de usos. El programa crea tres equipos, registra usos y produce un total.

SON DOS PROBLEMAS DISTINTOS Este ticket tiene un defecto de lógica y un defecto de memoria. Resuelva primero el de lógica: es el que destraba el código de cierre. El de memoria se verifica aparte, con el sanitizer.

Qué debe hacer

1. **Corregir el defecto de lógica.** El método registrar tiene una condición que deja fuera una posición del historial. Por eso EQ-03 suma 7 en vez de 8, y el total da 24 en vez de 25.
2. **Cerrar las fugas de memoria.** Hay tres puntos donde se reserva memoria y nunca se libera: el historial de cada Equipo, el arreglo de totales, y el arreglo de punteros a Equipo con los objetos que contiene. Busque los comentarios que dicen FALTA ALGO AQUI y FALTAN LIBERACIONES AQUI.

Criterio de aceptación

- El programa imprime SUMA=25 y a continuación el código de cierre.
- Compilado con -fsanitize=address, la ejecución NO produce ningún bloque que diga ERROR: AddressSanitizer.
- Se verifican las dos cosas en la mesa de despacho. Una sola no basta.

Cómo se verifica

```
$ cd prueba_corte1/estudiante/TCK-4421
$ g++ -std=c++17 -fsanitize=address -g -o inventario inventario.cpp
$ ./inventario
```

Si hay fugas, al final de la ejecución verá algo como esto:

Ejemplo de reporte de fugas (esto NO debe aparecer)

```
==509==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 48 byte(s) in 3 object(s) allocated from:
    #1 ... in main inventario.cpp:44
SUMMARY: AddressSanitizer: 228 byte(s) leaked in 8 allocation(s).
```

El detector le indica la línea exacta donde se pidió la memoria que no se liberó. Úselo: es la forma más rápida de encontrarlas.

Errores frecuentes en este ticket

- Usar delete en vez de delete[] para memoria reservada con new[].
- Liberar el arreglo de punteros antes de liberar los objetos que contiene: se pierden las direcciones y ya no se pueden liberar.
- Olvidar que la clase Equipo necesita un destructor que libere su propio historial.

Código de cierre obtenido: _____


```

#include <iostream>
#include <string>
using namespace std;

class Equipo {
private:
    string codigo;
    int* historial; // kilos/ usos registrados
    int n;
public:
    Equipo(string c, int cantidad) : codigo(c), n(cantidad) {
        historial = new int[n];
        for (int i = 0; i < n; i++) historial[i] = 0;
    }

    // Destructor para liberar la memoria del historial
    ~Equipo() {
        delete[] historial;
    }

    // Correccion de la condicion (i < n en lugar de i < n - 1)
    void registrar(int i, int v) { if (i >= 0 && i < n) historial[i] = v; }
    int total() const { int s = 0; for (int i = 0; i < n; i++) s += historial[i]; return s; }
    string getCodigo() const { return codigo; }
};

int* copiarTotales(Equipo** equipos, int cuantos) {
    int* copia = new int[cuantos];
    for (int i = 0; i < cuantos; i++) copia[i] = equipos[i]->total();
    return copia;
}

int main() {
    const int N = 3;
    Equipo** equipos = new Equipo*[N];
    equipos[0] = new Equipo("EQ-01", 4);
    equipos[1] = new Equipo("EQ-02", 4);
    equipos[2] = new Equipo("EQ-03", 4);

    equipos[0]->registrar(0, 5); equipos[0]->registrar(1, 3);
    equipos[1]->registrar(0, 9);
    equipos[2]->registrar(2, 7); equipos[2]->registrar(3, 1);

    int* totales = copiarTotales(equipos, N);
    int suma = 0;
    for (int i = 0; i < N; i++) {
        cout << equipos[i]->getCodigo() << ": " << totales[i] << endl;
        suma += totales[i];
    }
}

```

```
cout << "SUMA=" << suma << endl;

if (suma == 25) // el codigo se deriva de los totales correctos
    cout << "TICKET CERRADO - codigo de cierre: 4421-"
        << totales[0] << totales[1] << totales[2] << suma << endl;

// Liberacion de memoria para evitar fugas
delete[] totales;
for (int i = 0; i < N; i++) {
    delete equipos[i];
}
delete[] equipos;

return 0;
}
```

```
EQ-01: 8
EQ-02: 9
EQ-03: 8
SUMA=25
TICKET CERRADO - codigo de cierre: 4421-89825
PS C:\Users\Usuario\OneDrive\Desktop\Primer parcial practico> █
```

#la terminal

TCK-4422 El catálogo no distingue los tipos de recurso**SEVERIDAD P1** · PrestaLab · ventana 55–75 min · C++ · 1.1 puntos

Reportado por: Biblioteca comunitaria

«Necesitamos que el catálogo maneje libros y equipos, no solo recursos genéricos. De los libros interesa el autor; de los equipos, las horas de uso.»

«Ahora mismo el listado imprime Recurso genérico para todo.»

Contexto del sistema

El archivo catalogo.cpp tiene una clase Recurso y un arreglo de punteros a Recurso. Faltan las dos clases especializadas, y el listado no está mostrando la descripción propia de cada tipo.

Qué debe hacer

1. **Crear la clase LibroFísico**, que herede de Recurso y agregue el atributo autor. Su descripción debe devolver exactamente: Libro LF-002 de Borges
2. **Crear la clase Equipo**, que herede de Recurso y agregue el atributo horasUso. Su descripción debe devolver exactamente: Equipo EQ-003 (12h)
3. **Activar las dos líneas comentadas del catálogo** en main, para que se creen los tres objetos.
4. **Marcar al menos un recurso como prestado**, llamando al método prestar().
5. **Hacer que cada objeto responda con SU propia descripción** aunque se recorra el arreglo con punteros a la clase base. Si todos imprimen «Recurso generico», falta una palabra clave en la clase base.

Criterio de aceptación

- La salida contiene las tres descripciones, cada una con su formato propio.
- Al menos un recurso aparece marcado como [PRESTADO].
- Sin reportes del sanitizer. Recuerde que una clase base con clases hijas necesita destructor virtual.

Cómo se verifica

```
$ cd prueba_corte1/estudiante/TCK-4422
$ ./pruebas_t4.sh
```

(si da error de permisos: `bash pruebas_t4.sh`)

El texto de las descripciones se compara carácter por carácter. Respete mayúsculas, espacios y paréntesis exactamente como se indican arriba.

Errores frecuentes en este ticket

- Olvidar la palabra virtual en el método descripcion() de la clase base: los objetos hijos imprimen la descripción del padre.
- Olvidar el destructor virtual: el sanitizer reporta fuga al liberar por puntero a la base.
- Escribir «Equipo EQ-003 (12 h)» con espacio, o «12hrs»: el texto debe coincidir exacto.

Código de cierre obtenido: _____

TCK-4423 PRODUCCIÓN CAÍDA — la fila pierde personas**SEVERIDAD PO** · TurnoJusto · ventana 75–100 min · Python · 1.3 puntos

Reportado por: Mesa de ayuda · tres reportes en la última hora

*«Atendí al primero de la fila y desaparecieron todos.»**«Retiré a una persona del final y la fila sigue mostrándola.»**«La fila dice que tiene gente cuando está vacía.»***Contexto del sistema**

El archivo fila.py implementa la fila de espera de un punto de atención mediante una lista enlazada simple. Cada nodo es una persona con su número de turno y su nombre. Los métodos llegar, cuantos y listar funcionan bien.

El defecto está en el método retirar. Busque los comentarios que dicen «caso 1» y «caso 2».

Qué debe hacer

Corregir el método retirar(self, turno) para que maneje correctamente los tres casos de eliminación en una lista enlazada:

1. **Caso 1 — el elemento es la cabeza.** Al retirarlo, la cabeza debe pasar a ser el siguiente nodo, no None. Ese es el bug que borra toda la fila.
2. **Caso 2 — el elemento está en el medio o al final.** El nodo anterior debe pasar a apuntar al que sigue del que se elimina, saltándose el nodo retirado.
3. **Caso 3 — el elemento no existe.** Debe devolver False sin dañar la fila. Este caso ya funciona; verifique que siga funcionando después de sus cambios.

NO CAMBIE LA FIRMA El método debe seguir llamándose retirar y recibir un turno. El archivo de pruebas lo llama por su nombre. Tampoco modifique llegar, cuantos ni listar.

Criterio de aceptación

- Las siete verificaciones de pruebas_t5.py en OK.
- Retirar el primero deja el resto de la fila intacta.
- Retirar del medio y retirar el último funcionan.
- Retirar un turno inexistente devuelve False.
- Retirar el único elemento deja la fila con cero personas.
- Varios retiros encadenados funcionan sin corromper la fila.

Cómo se verifica

```
$ cd prueba_corte1/estudiante/TCK-4423
$ python3 pruebas_t5.py
```

Sugerencia de método

Antes de tocar el código, dibuje en el espacio de abajo tres nodos enlazados y marque con una flecha cuál referencia debe cambiar en cada uno de los tres casos. Este ticket se resuelve más rápido dibujando que probando.

Código de cierre obtenido: _____

Hoja de ruta del turno

Anote aquí el código de cierre de cada ticket que logre cerrar. Este es el documento que se entrega en la mesa de despacho.

Ticket	Sev	Sistema	Código de cierre obtenido	Puntos	Visto bueno
TCK-4419	P4	Interno	(en papel, dentro de este cuadernillo)	0.7	
TCK-4420	P3	RedAcopio		0.9	
TCK-4421	P2	PrestaLab		1.0	
TCK-4422	P1	PrestaLab		1.1	
TCK-4423	P0	TurnoJusto		1.3	

Hora de cierre de turno: _____ Total de puntos: _____

Firma de despacho: _____

Antes de entregar, verifique

- Escribió su nombre en la portada.
- Respondió los seis fragmentos del TCK-4419, aunque no haya alcanzado los demás.
- Copió cada código de cierre completo, tal como aparece en pantalla.
- Dejó los archivos guardados en su equipo, sin borrar nada.
- Tiene las pruebas listas para ejecutarlas delante de la profesora.